EPFL

Final Exam, Theory of Computation 2021

- Books, notes, communication, calculators, cell phones, computers, etc... are not allowed.
- Your explanations and proofs should be clear enough and in sufficient detail so that they are easy to understand and have no ambiguities.
- You are allowed to refer to material covered in lectures (but not exercises) including theorems without reproving them.
- **Do not touch until the start of the exam.**

Good luck!

Name: _____ **N° Sciper:** _____

Problem 1 / 15 points	Problem 2 / 20 points	Problem 3 / 20 points	Problem 4 / 15 points	Problem 5 / 15 points	Problem 6 / 15 points

Total / 100

1 *(15 pts)* **Quick-fire round.** Which of the following statements are true?

1. If A is a finite set, then A is regular.
2. The language $\{0^n : n \text{ is divisible by } 2021\}$ is regular.
3. If A is regular and $B \subseteq A$ then B is regular.
4. Let A be undecidable but recognisable. Then $\overline{A}$ is unrecognisable.
5. The language $\{\langle N, w\rangle : N \text{ is an NFA and } N \text{ accepts } w\}$ is undecidable.
6. If A and B are decidable, then $A \cap \overline{B}$ is decidable.
7. If $A \in \mathbf{P}$ then $A^* \in \mathbf{P}$ where $A^* = \{a_1 a_2 \cdots a_n : n \geq 0,\ a_i \in A\}$ is the Kleene star.
8. HALT $\leq_m$ SAT.
9. If $A \leq_m B$ then $\overline{B} \leq_m \overline{A}$.
10. k-SAT $\leq_p$ 3-SAT for any $k \geq 1$.
11. If $\mathbf{P} = \mathbf{NP}$ then $\mathbf{NP} = \mathbf{coNP}$.
12. If SAT $\in \mathbf{P}$ then $\mathbf{P} = \mathbf{NP}$.
13. If A and B are both **NP**-hard, then $A \leq_p B$.
14. The language $\{\langle n\rangle : n \in \mathbb{N} \text{ and } n \text{ is not a prime number}\}$ is in **NP**.
15. A circuit of size m can be written equivalently as a CNF formula of size $O(m^2)$.

For each box below, indicate whether the statement is either **T**rue, **F**alse, or if you are uncertain, leave the box empty. A correct answer is worth +1 point, an incorrect answer is worth −1 point, and an empty answer is worth 0 points.

Your answers:

1.	
2.	
3.	
4.	
5.	

6.	
7.	
8.	
9.	
10.	

11.	
12.	
13.	
14.	
15.	

Solution: True statements: 1,2,4,6,7,10,11,12,14.

2 *(20 pts)* **Basics of NP.**

2a Write down the definition of the class **NP** and the definition a given problem to be **NP**-complete. If you use the concept of a verifier or an NTM, then explain what it is.

Solution: A language is in NP if it has a polynomial-time verifier or equivalently a polynomial-time nondeterministic decider.

A polynomial-time verifier is a Turing Machine M such that given an input x, if x is in the language, then there exists a certificate C such that M accepts (x, C); if x is not in the language, then for every certificate C, M rejects (x, C).

A nondeterministic decider is an NTM N such that for each input x, every computation of N on x halts, and if x is in the language, then some computation of N on x accepts; if x is not in the language, then every computation of N on x rejects.

A problem L is **NP**-complete if $L \in \mathbf{NP}$ and for every $L' \in \mathbf{NP}$ we have $L' \leq_p L$, meaning that there is a polynomial-time computable function $f\colon \{0,1\}^* \to \{0,1\}^*$ such that $x \in L' \iff f(x) \in L$.

2b The class of problems solvable in deterministic exponential time is defined by

$$\mathbf{EXP} = \bigcup_{k \geq 1} \mathbf{TIME}(2^{n^k}).$$

Show that $\mathbf{NP} \subseteq \mathbf{EXP}$.

Solution: Exponential time allows a solution by exhaustive search of any problem in NP as follows. Given a problem in NP — that is, given a problem, its certificate-checking Turing machine, and its polynomial bound — we enumerate all possible certificates, feeding each in turn to the certificate-checking Turing machine, until either the machine answers "yes" or we have exhausted all possible certificates. The key to the proof is that all possible certificates are succinct, so that they exist "only" in exponential number.

Specifically, if the tape alphabet of the Turing machine has d symbols (including the blank) and the polynomial bound is described by $p()$, then an instance x has a total of $d^{p(x)}$ distinct certificates, each of length $p(x)$. Generating them all requires time proportional to $p(x) \cdot d^{p(x)}$, as does checking them all (since each can be checked in no more than $p(x)$ time). Since $p(|x|) \cdot d^{p(|x|)}$ is bounded by $2^{(q(|x|))}$ for a suitable choice of polynomial q, any problem in NP has a solution algorithm requiring at most exponential time.

3 *(20 pts)* **Regular languages.** For the following two languages, determine, with proof, whether they are regular. If a language is regular it suffices to draw a DFA or an NFA. You do not need to prove that your automaton is correct.

3a $A = \{0^n : n \text{ is a power of } 2\}$

3b $B = \{w \in \{0,1\}^* : w \text{ has an equal number of occurrences of 01 and 10 as substrings}\}$.

(For example, 010 *has one occurrence of* 01 *and one occurrence of* 10*, so* $010 \in B$*. On the other hand,* 01101 *has two occurrences of* 01 *and one occurrence of* 10*, so* $01101 \notin B$*.)*

Solution:

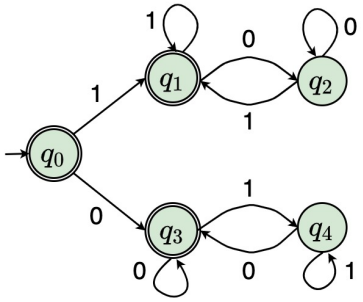
3a We will show that A is **not** regular. We assume towards a contradiction that A is regular.

We can apply the pumping lemma to A. Let p be the pumping length of A. Let $k := \min_i 2^i > p$. Then by definition $0^k \in A$ and $|0^k| > p$. Thus we can apply the pumping lemma to 0^k, which states that 0^k can be partitioned into $0^k = xyz$, such that $|xy| \leq p, |y| \geq 1$ and for all $i \in \mathbb{N}$ we have $xy^iz \in A$. Let us apply the lemma for $i = 2$. We bound the length of xy^2z from above and from below:

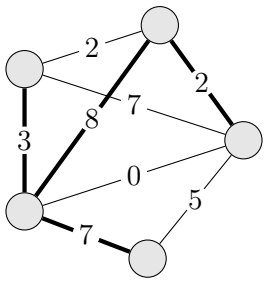
$$k = |xyz| < |xy^2z| = |xyz| + |y| = k + |y| \leq k + p < 2k,$$

where in the last inequality we used the definition of k. Thus we know that the length of xy^2z is strictly between two consecutive powers of 2 (i.e. $k, 2k$), which means that $xy^2z \notin A$. This, in turn, contradicts the statement of the pumping lemma, which implies that $xy^2z \in A$. We arrived at a contradiction, which means that A is not regular.

3b Yes, such a DFA exists as shown on the figure below. State q_0 is a start state and set of accept states is $\{q_0, q_1, q_3\}$.



4 *(15 pts)* **NP-completeness.** Let $G = (V, E)$ be a graph and $w\colon E \to \mathbb{N}$ an assignment of non-negative integer weights to the edges. A subset of edges $E' \subseteq E$ is a *spanning tree* if the subgraph (V, E') is a tree (no cycles) that connects all vertices. The weight of the spanning tree is $\sum_{e \in E'} w(e)$. For example, the bold edges below form a spanning tree of weight $3+7+8+2 = 20$.



In the Exact Spanning Tree problem (ESP for short), the input consists of a graph $G = (V, E)$, edge weights $w\colon E \to \mathbb{N}$, and a target $k \in \mathbb{N}$. The goal to decide whether G contains a spanning tree of weight exactly k. That is,

$$\text{ESP} \;=\; \{\langle G, w, k\rangle : G \text{ contains a spanning tree of weight exactly } k\}.$$

Show that ESP is **NP**-complete.

(You may use, without proof, the **NP***-completeness of any problem discussed in lectures.)*

Solution: We will first show that ESP is in **NP**. We can take the certificate to be the subset of edges of the graph. To check that these subset of edges form a spanning tree, one can use the breadth first search algorithm. To check that the sum of weights of these edges is k also requires going through these edges once and adding their weights. Both tasks in total take linear time in number of edges and vertices.
Now we show that ESP is **NP**-hard by reducing from Subset-Sum. In an instance $\langle S, t\rangle$ of Subset-Sum, where S is a set of non-negative integers and t is a non-negative integer, one has to decide whether $\exists$ a subset T of S that sums to t. We will convert an instance of Subset-sum into ESP as follows.

1. For each integer i in S, add two separate vertices v and v' to G. Add an edge of weight i between these two new vertices.
2. For every pair of vertices in G not connected by an edge, add an edge of weight 0.
3. Set $k = t$.

The reduction clearly takes polynomial time. Now we prove that $\langle S, t\rangle \in$ Subset-Sum iff $\langle G, k\rangle \in$ ESP.

1. If $\langle S, t\rangle \in$ Subset-Sum, $\exists$ a $T \subseteq S$ that sums to t. Hence, a spanning tree in G can be formed by taking edges corresponding to integers in T and then connecting the isolated vertices by zero edges.
2. If $\langle G, k\rangle \in$ ESP, the sum of edges selected in the spanning tree sum to k. By taking the integers corresponding to the selected non-zero edges in S, one gets T that sums to t. Hence, $\langle S, t\rangle \in$ Subset-Sum.

5 *(15 pts)* **Decidability.** Consider the language

$$L = \{\langle M \rangle : M \text{ is a Turing machine that halts on every input}\}$$

Classify L as one of (i) decidable, (ii) undecidable but recognisable, (iii) unrecognisable. Justify your answer with a proof.

Solution: Intuitively, L is harder than the halting language $HALT$, which is undecidable but recognizable. From the closure properties of recognizable languages we know that the complement $\overline{HALT}$ needs to be unrecognisable. We build a reduction ϕ from $\overline{HALT}$ to L.

$\phi(\langle M, w \rangle)$ is the Turing machine that on input x:

- Runs M on w for $|x|$ steps.
- If M halts then it loops.
- Otherwise, it halts.

We show correctness:

$\Rightarrow$) If $\langle M, w \rangle \in \overline{HALT}$ then M does not halt on w. Thus N halts on every input, so $\langle N \rangle \in L$.

$\Leftarrow$) If $\langle M, w \rangle \notin \overline{HALT}$ then M halts on w after some s steps. Then N accepts if and only if $x < s$. Therefore $\langle N \rangle \notin L$.

6 *(15 pts)* **Circuit complexity.** This problem asks you to show that regular languages admit linear-size circuits. That is, let $L \subseteq \{0,1\}^*$ be a regular language. For any input length $n \in \mathbb{N}$, show how to construct a boolean circuit C_n with n input variables, one output wire, and $O(n)$ gates such that

$$\forall x \in \{0,1\}^n : \quad C_n(x) = 1 \iff x \in L.$$

Solution: Let $\mathcal{D} = (Q, \{0,\,1\}, \delta, q_0, F)$ be a DFA that recognizes L. For simplicity, let us assume that the k states are labelled with $\{e_1, e_2, \ldots, e_k\}$ where $e_i \in \{0,\,1\}^k$ is the indicator bit-string which is 1 only at position i.

Note that the transition function $\delta : \{0,\,1\}^{k+1} \to \{0,\,1\}^k$ can be computed by a circuit Δ of constant size. Indeed, since the input size is $k+1$ and the output size is k, the function is described by a truth table of size $(2^k)^{2^{k+1}} \in O(1)$.

To construct the circuit that recognizes L on inputs of length n, we mimic $\mathcal{D}$ by processing the input iteratively, updating the state each time with Δ. When the last bit of the input has been processed, we check the final state and output 1 if and only if it belongs to F (this can be done with a constant number of gates), see Figure 1 for a depiction of the circuit. Since the circuit emulates $\mathcal{D}$, it correctly computes L while using $n \cdot O(1) + O(1) = O(n)$ gates.

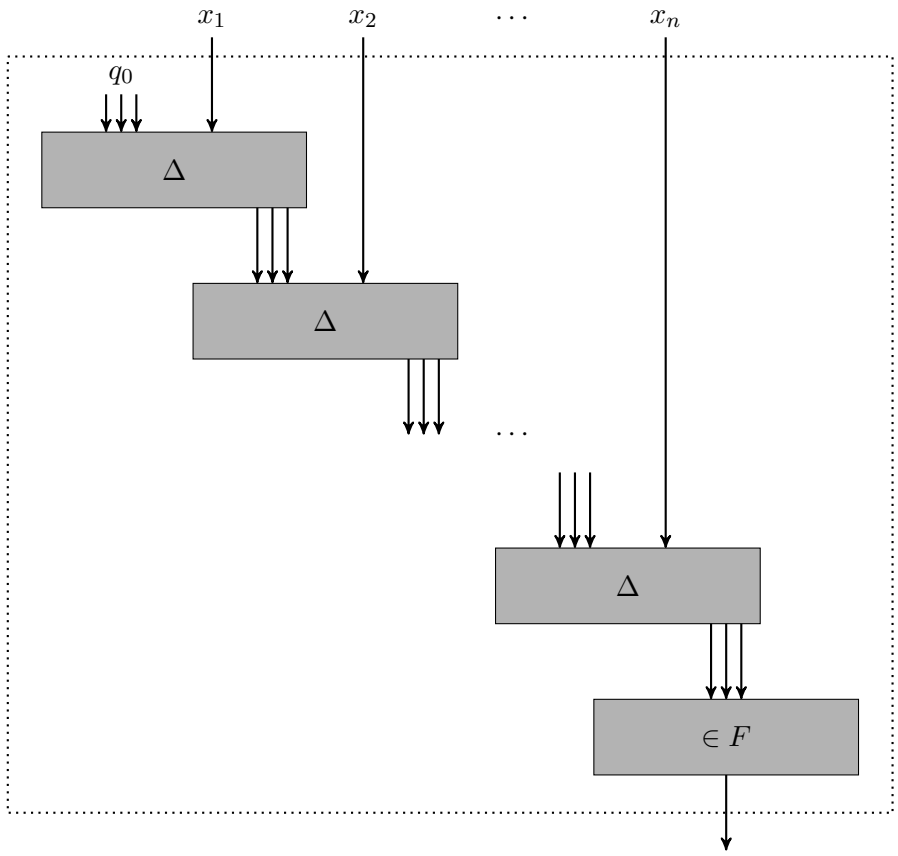


Figure 1. Architecture of the circuit that accepts n-bit words of L.